

Nexus Talent

Plataforma Web de Portafolios Digitales con Sistema de Mentoría y Retroalimentación entre Pares

Entrega Final del Proyecto de Software

Presentado por:

Nelson Andrés Ramírez Gutiérrez

Docente:

Tatiana Cabrera

Corporación Universitaria Iberoamericana

Programa de Desarrollo de Software

Bogotá, Colombia

2026

Introducción

El presente documento corresponde a la entrega final de la asignatura Proyecto de Software de la Corporación Universitaria Iberoamericana. En esta fase conclusiva, se documenta la implementación, despliegue y validación de Nexus Talent, una plataforma web orientada a solucionar la dispersión y falta de visibilidad del talento tecnológico y creativo multidisciplinar emergente en Colombia.

A lo largo de este documento se evidencia la transición desde el diseño conceptual y prototipado (desarrollado en las primeras entregas) hacia un entorno de producción real, aplicando el paradigma de la Programación Orientada a Objetos (POO), arquitectura Cliente-Servidor con React y Node.js, y un robusto esquema de integración y despliegue continuo (CI/CD). Además, se incluyen las evidencias de pruebas de software, la configuración de la infraestructura mediante contenedores y una retrospectiva analítica del trabajo bajo el marco de trabajo Scrum.

1. Cierre del Problema y Alcance del Proyecto

1.1 Descripción del Problema Inicial y la Solución

La problemática identificada radicaba en la ausencia de una plataforma integral que permitiera a estudiantes y profesionales jóvenes del ecosistema digital colombiano visibilizar su trabajo, recibir mentoría estructurada y obtener retroalimentación colaborativa. Nexus Talent resuelve este problema proporcionando un único entorno centralizado (construido con React en el frontend y Express/Node.js en el backend) donde los usuarios pueden gestionar portafolios verificables, superando la fragmentación que obligaba a utilizar múltiples herramientas desconectadas.

1.2 Comparación: Alcance Planeado vs. Ejecutado

El proyecto fue estructurado metodológicamente en cinco Sprints bajo la filosofía Scrum. A continuación, se detalla el nivel de cumplimiento del alcance:

Módulo (Sprint Planeado)	Estado de Ejecución	Observaciones
Autenticación y Perfil (Sprint 1)	100% Implementado	JWT, encriptación bcrypt, validaciones activas.
Portafolio Digital (Sprint 2)	100% Implementado	Gestión de proyectos con integración a la API de Cloudinary.
Retroalimentación (Sprint 3)	100% Implementado	Sistema de comentarios y sistema de puntuación en PostgreSQL.
Sistema de Mentoría (Sprint 4)	No Implementado	Se priorizó la estabilización del MVP Core.
Panel de Administración (Sprint 5)	No Implementado	Fuera del alcance técnico del semestre académico.

1.3 Justificación de Funcionalidades No Implementadas

La exclusión del Módulo de Mentoría (Sprint 4) y el Panel de Administración (Sprint 5) se derivó

de una decisión técnica enfocada en la calidad del software. La implementación de notificaciones bidireccionales concurrentes mediante Socket.io demandó mayor tiempo en la arquitectura del backend. Como desarrollador único (Solo Developer), se tomó la decisión estratégica de asegurar una cobertura de pruebas del 85% y un pipeline de despliegue estable para el Producto Mínimo Viable (MVP), antes de incorporar mayor complejidad técnica.

2. Pruebas de Software (Testing)

Garantizar la estabilidad de la plataforma fue primordial. Se utilizó **Jest** junto con **Supertest** para aislar el entorno de la API, y **Playwright** para la automatización del navegador, alcanzando una cobertura general del código (coverage) superior al 85%.

2.1 Pruebas de Integración (Backend)

1. **Flujo de Autenticación:** Verifica que el controlador procese correctamente un payload JSON, encripte la contraseña, guarde en PostgreSQL y retorne el código HTTP 201 junto con un JWT válido.
2. **Creación de Proyectos en Portafolio:** Confirma que un usuario logueado con un token JWT en las cabeceras pueda hacer POST a la ruta de proyectos y que las imágenes se deleguen adecuadamente a Cloudinary.
3. **Sistema de Retroalimentación:** Verifica que la base de datos no permita a un usuario dejar un comentario y valoración sobre su propio proyecto, asegurando las reglas del dominio de negocio.

2.2 Pruebas End-to-End (E2E con Playwright)

1. **Registro y Redirección:** El bot de Playwright ingresa credenciales en el formulario, valida que el botón de visibilidad de contraseña funcione, envía el formulario y verifica la redirección exitosa al Dashboard del Perfil.
2. **Gestión de Perfil Público:** Simulación de inicio de sesión, edición de habilidades separadas por comas, y comprobación visual de que los tags se renderizan correctamente en la interfaz de React.
3. **Interacción entre Pares:** Simulación de dos sesiones independientes para verificar que el comentario de un usuario "A" aparece de manera reactiva en el portafolio del usuario "B".

3. Pipeline de Integración y Despliegue Continuo (CI/CD)

Se implementó la automatización de la integración a través de **GitHub Actions** para garantizar la calidad en el código principal. El flujo de trabajo configurado se activa con cada *Pull Request* desde la rama develop hacia main.

3.1 Etapas del Pipeline

- **Build:** Ejecución de `npm run build` en el cliente React para prevenir rupturas en el empaquetado estático.
- **Linting:** Análisis estático del código fuente con ESLint para mantener convenciones limpias y legibilidad estándar de la industria.
- **Testing Automatizado:** Ejecución forzosa del conjunto de pruebas (`npm test`). Si el coverage disminuye o una aserción falla, el Pull Request es bloqueado.

Al completarse el *merge* exitosamente en la rama main, los webhooks configurados disparan de forma automática la compilación y el despliegue tanto en Render (Backend) como en Vercel (Frontend).

4. Aplicación en Producción y Contenedores (Docker)

4.1 Despliegue en la Nube

- **Frontend (React SPA):** Desplegado en Vercel, aprovechando su red de entrega de contenido (CDN) global.
- **Backend (Node.js/Express) y Base de Datos:** Alojados en Render, junto con el clúster de la base de datos PostgreSQL 15, garantizando la concurrencia y resiliencia de la API RESTful.

4.2 Entorno Local (Docker)

Cumpliendo con el requerimiento de aprendizaje de infraestructura, se crearon los archivos de contenerización. Manteniendo las decisiones de arquitectura, el punto de ejecución principal (`index.js`) se encuentra posicionado en la raíz del proyecto backend, fuera de la carpeta `src`.

Configuración Dockerfile (Backend)

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY . .
# El punto de ejecución se mantiene fuera de src por diseño de arquitectura
CMD ["node", "index.js"]
```

Configuración docker-compose.yml

```
version: '3.8'
services:
  nexus-api:
    build: ./backend
    ports:
      - "3001:3001"
    environment:
      - DB_HOST=postgres-db
      - DB_USER=${DB_USER}
      - DB_PASS=${DB_PASS}
    depends_on:
      - postgres-db
  postgres-db:
    image: postgres:15
    environment:
      - POSTGRES_USER=${DB_USER}
      - POSTGRES_PASSWORD=${DB_PASS}
      - POSTGRES_DB=nexustalent
    ports:
      - "5432:5432"
```

5. Documentación Técnica del Proyecto

La documentación fue tratada como un artefacto de primera clase para garantizar la mantenibilidad del sistema por futuros equipos.

- **README del Repositorio:** Instrucciones claras de instalación, requerimientos técnicos, stack de tecnologías y los comandos necesarios para desplegar los entornos tanto del frontend como del backend de forma local.
- **Documentación API (Swagger):** Una interfaz interactiva generada con swagger-jsdoc y expuesta en la ruta /api/docs del servidor, detallando esquemas, parámetros de entrada, tokens de autorización y respuestas HTTP para todos los endpoints.
- **JSDoc en el Código:** Funciones complejas, especialmente controladores de negocio y middlewares de encriptación y autenticación, cuentan con etiquetas y descripciones JSDoc directamente en el código fuente.

- **GitHub Wiki:** Se centralizaron los diagramas del modelo C4, el esquema Entidad-Relación y los Documentos de Registro de Decisiones Arquitectónicas (ADR) en la Wiki del repositorio principal.

6. Retrospectiva Final del Proyecto

El desarrollo de **Nexus Talent** bajo el marco de trabajo Scrum como "Solo Developer" representó un proceso inmenso de maduración técnica y organizativa.

Aprendizajes y Dificultades

El mayor desafío técnico inicial radicó en la configuración del entorno de base de datos local (PostgreSQL en Windows 11) debido a bloqueos de variables de entorno (PATH) que afectaron la agilidad inicial. Además, la gestión de políticas CORS y promesas asíncronas entre la SPA de React (puerto 3000) y la API de Express (puerto 3001) generó fricciones superadas mediante una adecuada parametrización de middlewares.

Si se reiniciara el proyecto, se optaría por implementar los contenedores Docker desde el Sprint 1 como entorno local estándar en lugar de esperar a las etapas finales, para evitar así las incidencias de compatibilidad originadas por el sistema operativo nativo.

Métricas del Proyecto

- **Velocity:** 100% de completitud en los Sprints 1 y 2. 85% de completitud en el Sprint 3.
- **Historias de Usuario:** 6 Historias de Usuario "Core" completadas y validadas contra criterios de aceptación de un total de 8 historias formuladas.
- **Avance del Backlog:** El backlog inicial quedó completado en un 75%, dejando un MVP funcional, robusto y preparado para escalamiento.

Conclusiones

El proceso de ingeniería de software desarrollado culminó exitosamente con la materialización de Nexus Talent. El uso metodológico de la arquitectura C4 y el desarrollo iterativo con Scrum garantizaron que la base tecnológica (PostgreSQL, Express, React, Node.js) no solo resolviera de manera efectiva la necesidad de portafolios verificables para jóvenes talentos, sino que constituyera una arquitectura escalable, segura (JWT/Bcrypt) y preparada para operaciones reales gracias a su pipeline automatizado de CI/CD.

Referencias Bibliográficas

Brown, S. (2018). *The C4 model for visualising software architecture*. <https://c4model.com>

Ministerio de Tecnologías de la Información y las Comunicaciones. (2023). *Informe del sector TIC en Colombia 2023*. MinTIC. <https://www.mintic.gov.co>

Organización Internacional de Normalización. (2011). *ISO/IEC 25010:2011 - Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - System and software quality models*. ISO.

Project Management Institute. (2021). *A guide to the project management body of knowledge (PMBOK® Guide) (7th ed.)*. Project Management Institute.

Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide: The definitive guide to Scrum - The rules of the game*. Scrum.org. <https://scrumguides.org/scrum-guide.html>